ShadowBoard AI Implementation Guide - Part 1

Part 1 of N: Account Setup & Project Scaffolding

Implementation Guide

How This Guide Is Structured

This is a multi-part series. ShadowBoard AI is a large system (7 AI agents, FastAPI backend, React frontend, Qdrant, PostgreSQL, Redis/Celery, Docker). Delivering it as one document with complete, non-placeholder code for every file would run to many hundreds of pages and would not be reliable to write in one pass. Instead, each part is a complete, working milestone you can run and verify before moving to the next.

Planned parts:

- **Part 1 (this document):** Account setup, full project folder structure, backend skeleton (FastAPI), frontend skeleton (React + Vite + TS), Docker Compose for Postgres/Redis/Qdrant, all environment/config files.
- **Part 2:** PostgreSQL models, JWT authentication, document upload API, MCP-based document ingestion.
- **Part 3:** Qdrant integration, Gemini embeddings, chunking and retrieval pipeline.
- **Part 4:** Google ADK agent framework setup + Moderator Agent + CFO Agent (full code, prompts, retrieval, error handling).
- **Part 5:** Product Strategy, Engineering, Marketing, Legal, Risk agents (full code).
- **Part 6:** A2A debate protocol, Lyzr Agent Studio workflow wiring, Celery background execution.
- **Part 7:** Frontend — dashboard, upload UI, live agent debate view, recommendation report, charts.
- **Part 8:** Logging, monitoring (Prometheus/Grafana), testing, Docker deployment, README.

Each part ends with **exact run/verify instructions** before you move on. Reply “Continue” after each part to receive the next one as a new PDF.

Step 1 — Create External Service Accounts

1.1 Objective

Obtain API credentials for Google Gemini, Qdrant, and Lyzr Agent Studio. Every backend service we build depends on these.

1.2 Why

Without these keys, no AI call, embedding, vector search, or workflow execution in this project will function. Getting them first avoids interruptions later.

1.3 Google Gemini API Key

1. Go to `https://aistudio.google.com/apikey`
2. Sign in with a Google account.
3. Click **Create API key**, select or create a Google Cloud project.
4. Copy the key (format `AIza...`).

Verification: run this once you have Python installed (Step 3 below covers installation):

```
curl "https://generativelanguage.googleapis.com/v1beta/models?key=YOUR_KEY"
```

Expected output: a JSON list of available Gemini models. If you get `API_KEY_INVALID`, the key was copied incorrectly or not yet activated — regenerate it.

1.4 Qdrant Cloud

1. Go to `https://cloud.qdrant.io` and sign up.
2. Click **Create Cluster** → choose the free tier.
3. Once provisioned, open the cluster and copy:
   - **Cluster URL** (e.g. `https://xxxx.cloud.qdrant.io`)
   - **API Key**

Alternative: we will also run a local Qdrant container via Docker Compose in Step 5, which needs no signup. You can use that for local development and switch to Cloud for deployment — both will be configured via the same `.env` variable, so nothing later changes.

1.5 Lyzr Agent Studio

1. Go to `https://studio.lyzr.ai` and sign up.
2. Navigate to **Settings → API Keys** (or **Developer** section).
3. Generate and copy an API key.

1.6 Common Errors

Error	Cause	Solution
`API_KEY_INVALID` (Gemini)	Key copied with trailing space, or project billing not enabled	Re-copy exactly; enable billing/quota in Cloud Console if prompted
Qdrant cluster stuck “Provisioning”	Free tier cold start	Wait 1–2 minutes, refresh dashboard
Lyzr key not visible	Not verified email	Verify email first, then revisit API Keys page

1.7 Checklist before continuing

- ☐ Gemini API key saved somewhere safe
- ☐ Qdrant Cluster URL + API key saved (or decided to use local Docker Qdrant only)
- ☐ Lyzr API key saved

Step 2 — Install Local Toolchain

2.1 Objective

Install all software needed to run the project locally: Git, Python 3.11+, Node.js 20+, Docker Desktop.

2.2 Why

The backend runs on Python/FastAPI, the frontend on Node/Vite, and Postgres/Redis/Qdrant run as Docker containers so you don’t need to install databases natively.

2.3 Install Git

- Windows: `https://git-scm.com/download/win`

- Mac: `brew install git`
- Linux: `sudo apt install git`

Verify:

```
git --version
```

Expected: `git version 2.x.x`

2.4 Install Python 3.11+

- Windows/Mac: `https://www.python.org/downloads/` (check “Add to PATH” on Windows)
- Linux: `sudo apt install python3.11 python3.11-venv python3-pip`

Verify:

```
python3 --version
```

Expected: `Python 3.11.x` or higher.

Common error: `python3: command not found` on Windows → use `python --version` instead, since the Windows installer registers it as `python`.

2.5 Install Node.js 20+

Download from `https://nodejs.org` (LTS version) or use `nvm`:

```
nvm install 20
nvm use 20
```

Verify:

```
node --version
npm --version
```

Expected: `v20.x.x` and an npm version `10.x.x`.

2.6 Install Docker Desktop

- Download from `https://www.docker.com/products/docker-desktop/`
- Install and start it (it must be running in the background for later steps).

Verify:

```
docker --version
docker compose version
```

Expected: version numbers for both, no errors.

Common error: `Cannot connect to the Docker daemon` → Docker Desktop is not running; open the application and wait for the whale icon to show “running.”

Step 3 — Create the Project Folder Structure

3.1 Objective

Create the root project folder and the full directory skeleton for backend, frontend, and infrastructure files, matching what every later part will populate.

3.2 Terminal

Open your regular terminal (Terminal on Mac/Linux, PowerShell or Git Bash on Windows). Navigate to wherever you keep projects, e.g.:

```
cd ~/projects
```

3.3 Commands

```
mkdir shadowboard-ai
cd shadowboard-ai
git init

mkdir -p backend/app/agents
mkdir -p backend/app/api/routes
mkdir -p backend/app/core
mkdir -p backend/app/db/models
mkdir -p backend/app/schemas
mkdir -p backend/app/services/mcp
mkdir -p backend/app/services/qdrant_client
mkdir -p backend/app/services/lyzr
mkdir -p backend/app/services/a2a
mkdir -p backend/app/middleware
mkdir -p backend/app/tasks
mkdir -p backend/app/utils
mkdir -p backend/tests
mkdir -p backend/logs

mkdir -p frontend/src/components
mkdir -p frontend/src/pages
mkdir -p frontend/src/hooks
mkdir -p frontend/src/lib
mkdir -p frontend/src/types
mkdir -p frontend/src/store
mkdir -p frontend/public

mkdir -p infra/docker
mkdir -p docs
```

3.4 Expected Output

No errors printed. Running `find . -type d | sort` (Mac/Linux) or `tree /F` (Windows, if installed) should show the nested folders above.

3.5 Verification

```
ls backend/app
```

Expected output:

```
agents  api  core  db  middleware  schemas  services  tasks  utils
```

3.6 Common Errors

Error	Cause	Solution
`mkdir: cannot create directory: File exists`	Ran the block twice	Harmless, ignore — folders already exist
`git init` says “Reinitialized”	Already a repo	Harmless

Step 4 — Backend Skeleton (FastAPI)

4.1 Objective

Set up a Python virtual environment, install backend dependencies, and create a minimal FastAPI app that runs and responds, before any business logic is added.

4.2 Why

We verify the backend boots cleanly first. Every later part (auth, agents, Qdrant) adds to this skeleton rather than us discovering environment problems after 50 files exist.

4.3 Terminal

From the `shadowboard-ai` root:

```
cd backend
python3 -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
```

Expected output: your terminal prompt is now prefixed with `(venv)`.

4.4 Create `backend/requirements.txt`

```
fastapi==0.115.6
uvicorn[standard]==0.34.0
pydantic==2.10.4
pydantic-settings==2.7.1
python-dotenv==1.0.1
sqlalchemy==2.0.36
asyncpg==0.30.0
psycopg2-binary==2.9.10
alembic==1.14.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
python-multipart==0.0.20
celery==5.4.0
redis==5.2.1
qdrant-client==1.12.1
google-genai==0.3.0
loguru==0.7.3
httpx==0.28.1
tenacity==9.0.0
prometheus-fastapi-instrumentator==7.0.0
pytest==8.3.4
pytest-asyncio==0.25.0
```

Install:

```
pip install -r requirements.txt
```

Common error: `psycopg2-binary` fails to build on Mac M1/M2 → run `pip install psycopg2-binary --no-binary :all:` or simply `brew install postgresql` first, then retry the normal install.

4.5 Create `backend/app/core/config.py`

```
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    # App
    APP_NAME: str = "ShadowBoard AI"
    ENVIRONMENT: str = "development"
    DEBUG: bool = True

    # Auth
    JWT_SECRET_KEY: str
    JWT_ALGORITHM: str = "HS256"
    JWT_EXPIRE_MINUTES: int = 60 * 24

    # Database
    DATABASE_URL: str

    # Redis / Celery
    REDIS_URL: str = "redis://localhost:6379/0"
    CELERY_BROKER_URL: str = "redis://localhost:6379/1"
    CELERY_RESULT_BACKEND: str = "redis://localhost:6379/2"

    # Gemini
    GEMINI_API_KEY: str
    GEMINI_MODEL: str = "gemini-2.5-pro"
    GEMINI_EMBEDDING_MODEL: str = "text-embedding-004"

    # Qdrant
    QDRANT_URL: str
    QDRANT_API_KEY: str | None = None
    QDRANT_COLLECTION: str = "shadowboard_documents"

    # Lyzr
    LYZR_API_KEY: str
    LYZR_BASE_URL: str = "https://api.lyzr.ai"

    # CORS
    CORS_ORIGINS: str = "http://localhost:5173"


settings = Settings()
```

4.6 Create `backend/app/main.py`

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from loguru import logger

from app.core.config import settings

app = FastAPI(
    title=settings.APP_NAME,
    debug=settings.DEBUG,
    version="0.1.0",
)
```

```
    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.CORS_ORIGINS.split(","),
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )


    @app.on_event("startup")
    async def on_startup() -> None:
        logger.info(f"{settings.APP_NAME} starting in {settings.ENVIRONMENT} mode")


    @app.get("/health")
    async def health() -> dict:
        return {"status": "ok", "service": settings.APP_NAME}


    @app.get("/")
    async def root() -> dict:
        return {"message": "ShadowBoard AI backend is running. See /docs for API."}
```

4.7 Create `backend/app/__init__.py` (empty file)

```
```

4.8 Create `backend/.env.example`

```
ENVIRONMENT=development
DEBUG=true

JWT_SECRET_KEY=change_this_to_a_random_64_char_string
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440

DATABASE_URL=postgresql+asyncpg://shadowboard:shadowboard@localhost:5432/shadowboard

REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL=redis://localhost:6379/1
CELERY_RESULT_BACKEND=redis://localhost:6379/2

GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=gemini-2.5-pro
GEMINI_EMBEDDING_MODEL=text-embedding-004

QDRANT_URL=http://localhost:6333
QDRANT_API_KEY=
QDRANT_COLLECTION=shadowboard_documents

LYZR_API_KEY=your_lyzr_api_key_here
LYZR_BASE_URL=https://api.lyzr.ai

CORS_ORIGINS=http://localhost:5173
```

Now copy it to a real `.env` and fill in your real keys from Step 1:

```
    cp .env.example .env
```

Open `.env` in a text editor and paste in your Gemini key, Qdrant URL/key, and Lyzr key. Generate a random JWT secret with:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

Paste that value into `JWT_SECRET_KEY`.

4.9 Run the Backend

```
uvicorn app.main:app --reload --port 8000
```

Expected output:

```
INFO:     Uvicorn running on http://127.0.0.1:8000
INFO:     Application startup complete.
```

4.10 Verify

Open `http://localhost:8000/health` in a browser, or:

```
curl http://localhost:8000/health
```

Expected: `{"status":"ok","service":"ShadowBoard AI"}`

Also open `http://localhost:8000/docs` — you should see the interactive FastAPI Swagger UI.

4.11 Common Errors

Error	Cause	Solution
`pydantic_core._pydantic_core.ValidationError ... field required`	`.env` missing or values not filled	Confirm `.env` exists in `backend/` (not root) and all required keys are filled
`ModuleNotFoundError: No module named 'app'`	Running uvicorn from wrong directory	Run the command from inside `backend/`, not project root
`Address already in use`	Port 8000 taken	`uvicorn app.main:app --reload --port 8001`

Step 5 — Infrastructure: Docker Compose (Postgres, Redis, Qdrant)

5.1 Objective

Run PostgreSQL, Redis, and a local Qdrant instance as Docker containers so the backend has everything it needs without manual installs.

5.2 Create `infra/docker/docker-compose.yml`

```
version: "3.9"

services:
  postgres:
    image: postgres:16-alpine
    container_name: shadowboard_postgres
    restart: unless-stopped
    environment:
```

```
      POSTGRES_USER: shadowboard
      POSTGRES_PASSWORD: shadowboard
      POSTGRES_DB: shadowboard
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shadowboard"]
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: shadowboard_redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 5s
      retries: 5

  qdrant:
    image: qdrant/qdrant:latest
    container_name: shadowboard_qdrant
    restart: unless-stopped
    ports:
      - "6333:6333"
      - "6334:6334"
    volumes:
      - qdrant_data:/qdrant/storage

volumes:
  postgres_data:
  qdrant_data:
```

5.3 Run It

From the project root:

```
cd infra/docker
docker compose up -d
```

Expected output: three lines ending in `Started` or `Healthy`, e.g.

```
✔ Container shadowboard_postgres  Started
✔ Container shadowboard_redis     Started
✔ Container shadowboard_qdrant    Started
```

5.4 Verify

```
docker ps
```

Expected: three containers listed, `STATUS` showing `Up ... (healthy)` for postgres/redis.

Check Qdrant:

```
curl http://localhost:6333/healthz
```

Expected: `healthz check passed`

Check Postgres:

```
docker exec -it shadowboard_postgres psql -U shadowboard -d shadowboard -c "SELECT 1;"
```

Expected: a result row showing `1`.

5.5 Common Errors

Error	Cause	Solution
`port is already allocated` (5432/6379/6333)	Another Postgres/Redis/Qdrant already running locally	Stop the conflicting service, or change the left-hand port mapping, e.g. `"5433:5432"`, and update `DATABASE_URL` accordingly
`Cannot connect to the Docker daemon`	Docker Desktop not running	Start Docker Desktop, wait for it to fully boot, retry

5.6 Note on Qdrant Cloud vs Local

If you'd rather use Qdrant Cloud (from Step 1.4) instead of this local container, just don't run the `qdrant` service, and set in `.env`:

```
QDRANT_URL=https://xxxx.cloud.qdrant.io
QDRANT_API_KEY=your_cloud_api_key
```

Everything downstream is identical either way.

Step 6 — Frontend Skeleton (React + Vite + TypeScript)

6.1 Objective

Scaffold the frontend with Vite, TypeScript, Tailwind CSS, and verify it renders a starter page, before building real UI in Part 7.

6.2 Terminal

From the project root:

```
cd frontend
```

6.3 Initialize Vite Project

Since the `frontend` folder already exists with subfolders from Step 3, initialize Vite into it directly:

```
npm create vite@latest . -- --template react-ts
```

When prompted "Current directory is not empty. Remove existing files and continue?", choose **No** (we already created `src/components` etc. on purpose) — if Vite refuses to proceed, instead run:

```
npm create vite@latest temp-frontend -- --template react-ts
```

```
cp -r temp-frontend/* temp-frontend/.[!.]* . 2>/dev/null
rm -rf temp-frontend
```

6.4 Install Dependencies

```
npm install
npm install tailwindcss @tailwindcss/postcss postcss autoprefixer
npm install react-router-dom @tanstack/react-query axios zustand
npm install framer-motion recharts lucide-react clsx tailwind-merge
npx tailwindcss init -p
```

Expected output: `package.json` and `package-lock.json` updated, no red error text. Warnings about deprecated sub-dependencies are normal and safe to ignore.

6.5 Create `frontend/tailwind.config.js`

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: {
    extend: {
      colors: {
        brand: {
          50: "#eef2ff",
          500: "#4f46e5",
          600: "#4338ca",
          900: "#1e1b4b",
        },
      },
    },
  },
  plugins: [],
};
```

6.6 Create `frontend/src/index.css`

```
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  color-scheme: light dark;
}

body {
  margin: 0;
  font-family: "Inter", system-ui, sans-serif;
}
```

6.7 Create `frontend/src/lib/api.ts`

```
import axios from "axios";

export const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || "http://localhost:8000",
  headers: {
    "Content-Type": "application/json",
  },
});
```

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("shadowboard_token");
  if (token && config.headers) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

6.8 Create `frontend/src/App.tsx`

```
function App() {
  return (
    <div className="min-h-screen flex items-center justify-center bg-slate-50">
      <div className="text-center">
        <h1 className="text-3xl font-semibold text-brand-600">
          ShadowBoard AI
        </h1>
        <p className="mt-2 text-slate-500">
          Frontend scaffold running. Backend health check and real pages
          arrive in later parts.
        </p>
      </div>
    </div>
  );
}

export default App;
```

6.9 Create `frontend/.env.example`

```
VITE_API_BASE_URL=http://localhost:8000
```

```
cp .env.example .env
```

6.10 Run It

```
npm run dev
```

Expected output:

```
VITE v6.x.x  ready in xxx ms
➜  Local:   http://localhost:5173/
```

6.11 Verify

Open `http://localhost:5173` in your browser. You should see “ShadowBoard AI” centered on a light gray background.

6.12 Common Errors

Error	Cause	Solution
`Cannot find module 'tailwindcss'`	Install step skipped	Re-run the `npm install tailwindcss ...` line from 6.4
		Confirm `frontend/src/main.tsx` has

Blank white page, console shows CSS error	`index.css` not imported in `main.tsx`	`import "./index.css"` at the top (Vite scaffolds this by default — don't remove it)
Port 5173 already in use	Another Vite app running	`npm run dev -- --port 5174`

Step 7 — Root-Level Project Files

7.1 Create `.gitignore` (project root)

```
# Python
venv/
__pycache__/
*.pyc
backend/.env

# Node
node_modules/
frontend/.env
frontend/dist/

# Docker
infra/docker/volumes/

# Logs
backend/logs/*.log

# OS
.DS_Store
Thumbs.db
```

7.2 Create root `README.md`

```
# ShadowBoard AI

Enterprise Multi-Agent Decision Intelligence Platform.

## Structure
- `backend/` — FastAPI app, AI agents, services
- `frontend/` — React + Vite + TypeScript UI
- `infra/docker/` — Docker Compose for Postgres, Redis, Qdrant

## Local Development
1. Start infra: `cd infra/docker && docker compose up -d`
2. Start backend: `cd backend && source venv/bin/activate && uvicorn app.main:app --reload`
3. Start frontend: `cd frontend && npm run dev`

Full setup instructions: see the implementation guide PDFs (Parts 1–8).
```

7.3 Commit the Skeleton

```
cd shadowboard-ai
git add .
git commit -m "Part 1: project scaffolding, backend/frontend skeletons, docker infra"
```

Part 1 — Final Verification Checklist

Run all three of these simultaneously (separate terminals) and confirm each:

1. **Infra:** `docker ps` → postgres, redis, qdrant all `Up`
2. **Backend:** `curl http://localhost:8000/health` → `{"status":"ok",...}`
3. **Frontend:** `http://localhost:5173` in browser → “ShadowBoard AI” heading visible

If all three pass, the scaffold is solid and ready for Part 2: **PostgreSQL models, JWT authentication, and the document upload API.**

Reply **“Continue”** to receive Part 2.